2026 中国高校计算机大赛—人工智能创意赛

鸿蒙赛道作品说明文档

参赛学校：_____同济大学_____
团队名称：_____微软大战代码_____
作品名称：_____ 智调鸿枢：面向鸿蒙生态的 AI 端侧感知调度中间件__
赛题方向：_____操作系统智能创新_____

联系人（队长）：_____方鹭洋_____
联系电话（队长）：___18856857376_____

作品说明文档
包含但不限于以下内容：

- 参赛团队信息表
- 作品原创性声明
- 创意描述（100 字以内）
- 设计稿/技术方案
- 介绍文档（1500 字以内）

2026 中国高校计算机大赛—人工智能创意赛
参赛团队信息表

作品名称	智调鸿枢：面向鸿蒙生态的 AI 端侧感知调度中间件
团队名称	微软大战代码
参赛学校	同济大学
参赛赛题方向	（）应用创新 （）Agent 创新 （）用户体验创新 （√）操作系统智能创新

团队队员基本信息（可跨校、跨专业组队）

姓名	学校全称	院（系）全称	专业全称	年级	毕业时间	联系电话	邮箱	团队分工
方鹭洋	同济大学	计算机学院	软件工程	大三	2028.6	18856857376	2454329@tongji.edu.cn	队长
李一芃	同济大学	计算机学院	软件工程	大三	2028.6	19905688857	2454326@tongji.edu.cn	队员

团队指导教师信息（指导老师须与队长同校）

姓名	院（系）全称	职称	研究方向	联系电话	联系邮箱
金博	计算机学院	副教授	深度学习、自然语言处理	18856857376	bjin@tongji.edu.cn

团队成员优势描述

方鹭洋，同济大学软件工程本科生，具备 AI 算法、后端开发与系统架构能力，主导智能比价系统并被评为字节跳动优秀项目，参与磁浮探伤强化学习系统研究，同时具备机器学习系统相关设计经验

李一芃，同济大学软件工程本科生，在多模态大模型（VLM / LLM）应用开发、Agent 工程化等均有工程经验； 了解 PyTorch、Transformers 等主流深度学习框架，精通计算机系统结构、操作系统；曾获中兴捧月 AI 挑战赛区域优胜奖

项目分工：

方鹭洋：全链路设计与协议定版；手机端 apps/harmony（HAR 调度与 App）、搭建与 Artifact 存储与图片数据通道、真机与多设备测试微调、项目文档撰写。

李一芃：服务端与业务链路 services/api-server（12 阶段引用化与提交屏障、设备目录与执行适配器/worker），本地算法，服务端回归与 Zeabur 部署。

《 智调鸿枢：面向鸿蒙生态的端侧 AI 感知调度中间件 》 作品原创性声明

郑重声明：承诺本参赛队伍报名信息真实有效；呈交的参赛作品相关资料以及所完成的作品实物等相关成果，是本团队独立进行研究工作所取得的成果，除文中已经注明引用的内容外，本作品说明文档不包含任何其他个人或集体已经发表或撰写过的作品成果，不侵犯任何第三方的知识产权或其他权利。本声明的法律结果由本参赛队承担。

参赛队员签名（团队全部成员）：

日期：2026 年 9 月 28 日

指导老师审核签名：

日期：2026 年 9 月 28 日

（注：本页签名可以打印纸质文件后签名，提供扫描件；或者使用电子签名。不论哪种方式，需保证页面内容完整、字迹清晰，请与本项目创意书作为同一文档提交，否则项目创意书提交可能无效。）

作品说明文档提交规范说明

将综合考察评估各参赛队的**创新性、完备性、前景评估、规范性、实际应用价及团队成员背景及能力互补**。因此，为了更加清晰了解各参赛队的基本情况及作品内容，各参赛队（请以团队为单位）提交的《作品说明文档》将用于评委评审打分的核心依据。

一、 作品说明文档，具体内容：

（一） 创意描述

面向鸿蒙多设备协同，将手机、可信计算节点与云服务组织为统一任务资源视图，系统采用全局放置与本地自治的两级协同调度，并通过数据位置感知、质量约束与跨设备提交一致性，把资源治理从单机拓展到多设备。

（二） 技术方案（操作系统智能）

1. 设计目标与总体定位

传统调度以 CPU、内存与 I/O 等通用指标为依据，在进程与线程层面分配资源，不感知任务的语义与质量需求。对 AI 负载而言，模型精度、时延预算、能耗与热稳定性等关键诉求因此难以同硬件及操作系统的调度结果高度融合：调度器既不了解任务需要何种精度、容忍多大时延，也无法把硬件后端、模型档位与线程配置同任务目标对齐，往往造成“资源已经分配、结果却不满足需求”的错配。单设备治理还受限于本机资源，当本机算力、内存或热状态不足以承载某类 AI 任务时，任务只能被拒绝、降级或排队等待，而其他设备可能拥有更充裕的资源环境却无法被纳入调度；单机调度既无法感知其他设备的资源余量与数据位置，也无法把任务卸载到更合适的设备上执行。

针对上述不足，智调鸿枢将资源治理范围从单机的进程与线程，扩展到跨设备的任务、数据依赖与执行结果。治理对象不再只是单机内部的资源分配，而是工作流每个阶段的执行设备选择、依赖数据的存放位置，以及跨设备提交的一致性维护，从而把调度从单点资源分配提升为面向任务全生命周期的协同治理。

从机制定位上看，智调鸿枢将手机、可信计算节点与云服务纳入统一的能力目录：应用首先声明业务需求（精度下限、时延预算与隐私约束…），全局调度器依据数据位置、端到端成本与质量约束为每阶段选择执行设备，各设备的本地调度器负责本机的资源配置、准入与防震荡机制。设备的协作由此从连接与数据互通，进一步延伸到任务级的联合执行，使多台设备在同一任务身份与预算约束下共同完成一条工作流，并保持跨设备提交的一致性。

项目海报


智联万物
调度未来
HarmonyOS
万物智联
共创智能未来
智调鸿枢
鸿蒙
面向鸿蒙应用生态的端侧 AI 感知调度中间件
让 AI 任务感知设备状态，在手机、云端与可信计算设备之间有据可循地执行
参赛学校：同济大学
团队名称：微软大战代码
01 智能识物购物
文字 / 图片找商品
智能推荐与比价
02 资源感知调度
感知电量、性能、网络
智能决策执行位置
03 多设备能力派发
云端与可信设备的同
能力匹配，高效执行
04 Task + Artifact 闭环
任务拆解 · 阶段处理
结果可追踪
05 实时可观测
电脑调度盘可视化
过程可靠、可查
搜索想要的商品...
今日推荐
AI 识物购物
鸿蒙 App
文字 / 图片输入
商品结果
任务请求
Task + Artifact
智调鸿枢
Scheduler HAR
资源感知 · 任务调度
生命周期管理
智能评估
多端派发
云端 Runtime
推理服务 · 高品质
向量检索 · 图片处理
可信计算设备
CPU Worker
图片处理 · 向量归一化
设备测测
(仅观测)
执行回执
结果回传
电脑调度盘
实时观测 · 策略依赖 · 任务结果
应用价值
更流畅
合理调度，前台不等待
更高效
设备协同，释放算力
更安全
隐私可控，可信执行
更开放
可集成的 HarmonyOS 调度能力
HUAWEI

2. 智调鸿枢总体架构

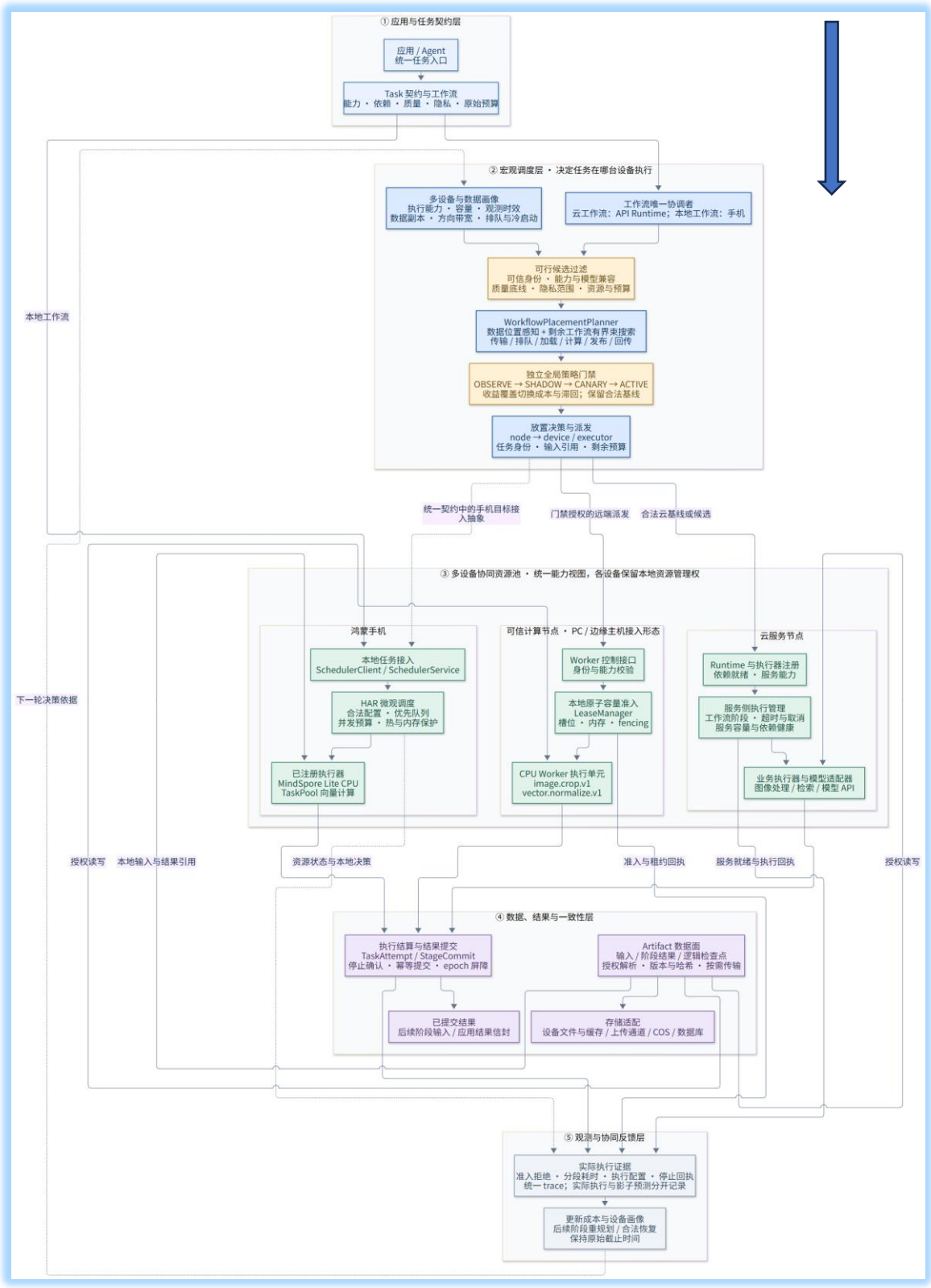


图 1　智调鸿枢总体架构

智调鸿枢以五层架构组织跨设备调度：应用与任务契约层把应用请求统一为 Task 契约

与工作流；宏观调度层作为工作流上层的唯一协调者，决定任务在哪台设备执行；多设备协同资源池提供统一的能力视图，同时各设备保留本地资源管理权。数据、结果与一致性层以 Artifact 数据面与阶段提交承载输入、结果和逻辑检查点；观测与协同反馈层则是负责利用实际执行证据持续更新成本与设备画像，并驱动后续阶段的调整与重规划。

2.1 任务与数据契约

最终架构以两类契约把调度核心与业务解耦，并在表示层区分“计算”与“数据”：Task 表达计算，Artifact 表达输入、阶段输出与逻辑检查点。契约分为静态与动态两类——静态契约声明能力、输入输出 schema、模型兼容条件、质量底线、合法执行配置与副作用策略；动态实例则携带工作流、节点、任务、尝试与编排器身份，并记录剩余预算与请求。为保证两端语义一致，调度协议通过共享样例加以验证；购物业务的 16 个数据家族再从同一来源分别生成 TS 与 ArkTS 类型及校验器。两类契约各自独立演进，使调度核心始终保持业务无关。

Artifact 以身份、版本、内容哈希、依赖和状态描述数据，控制消息携带小型引用，数据面独立承担内容传输、存储与授权访问，由授权解析器按需获取实际内容；签名 URL 在使用时生成，避免把短期访问凭证当成数据身份。内容提交前保留 STAGING 状态，校验后发布 COMMITTED 引用，阻止半成品进入后续节点。

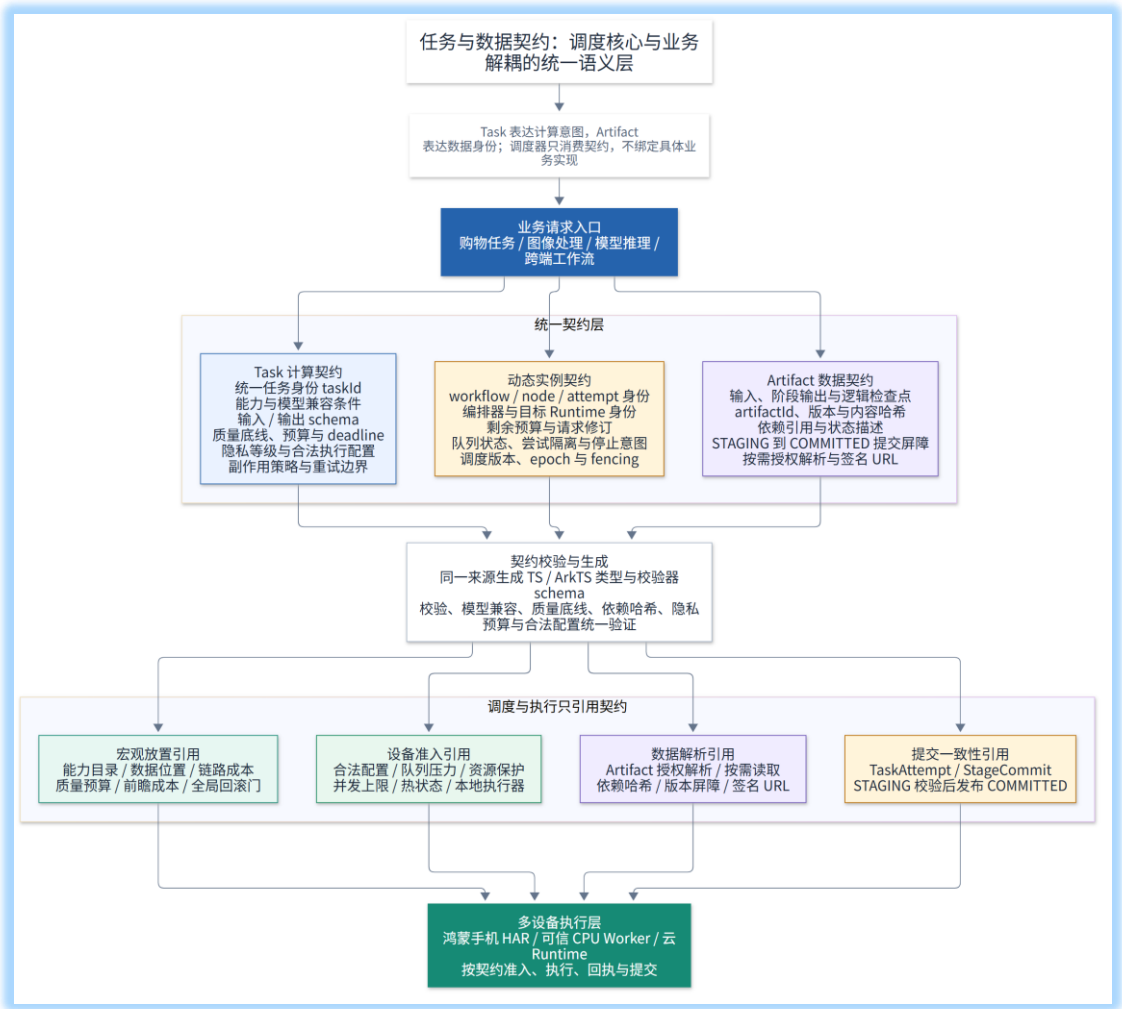


图 2　任务与数据契约驱动的统一语义层

组成	核心职责
任务与数据契约	统一任务身份、质量与预算；校验 Artifact 版本及依赖
多设备全局协调	维护任务图与能力目录；比较设备及数据位置成本
设备本地调度	选择合法配置；治理线程、批量、队列与资源占用
可信执行与提交	目标容量准入、停止回执、尝试隔离和结果提交
观测与校准	区分预测与事实；关联分段时延、结果归因码和策略版本
Artifact 数据面与权限	负责上传、快照、版本与哈希校验；按授权生成签名 URL，隔离控制消息与内容传输并保护敏感数据
预算租约与取消治理	下传剩余预算并执行目标原子准入；管理 lease/fence、停止确认、迟到回执和资源隔离，避免重复占用
恢复与提交一致性	以阶段检查点和不可变引用支持恢复或重算
统一观测与灰度回滚	贯通 trace、阶段时延、失败原因与策略版本；区分实际和影子执行，按 OBSERVE/SHADOW/CANARY/ACTIVE 灰度并可一键回退

表 1　最终架构的职责划分

2.2 全局统一放置与设备本地自治

全局层决定工作流的节点在哪台设备执行，本地层则是对接入设备进行实时调度。全局层依据设备与数据画像筛选可行候选，再结合数据位置与传输成本选择放置，交由目标设备准入执行，从而完成跨设备的阶段分配。本地层在目标设备收到任务后，依据电量、热等级、内存与队列等状态，在已验证的配置包之间确定后端、档位与线程。云端工作流由 API Runtime 协调，智调鸿枢具备一个全局调度器，目标设备拥有重新规划权限，避免同一阶段被重复派发。

设备接入按执行能力、输入类型、模型与索引进行兼容性登记，而不是依赖固定品牌或板卡。手机后端、可信 CPU Worker 和模型服务分别声明其真实能力；CPU、GPU、NPU 是配置候选，真实接入前我们设置了验证关卡，只有经可用性与执行验证后的配置才纳入最终选择。外部模型服务的硬件不计作手机或 Worker 的本地后端。

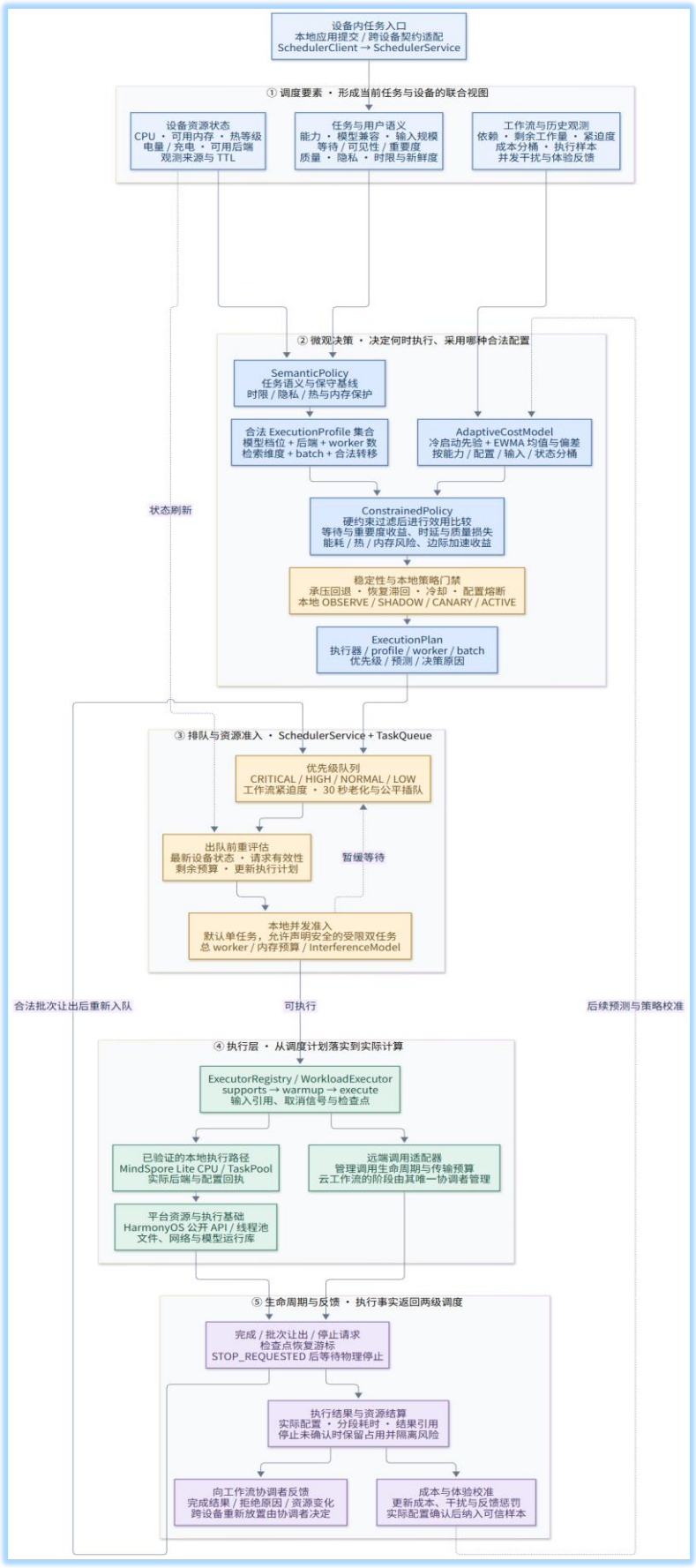


图 3　单设备微观调度架构

2.3 执行、数据与提交闭环

Runtime 按依赖顺序调用已注册的执行器，完成 AI 任务工作流的 12 个阶段。各阶段将结果保存为不可变的 Artifact，供后续阶段通过引用读取。 WorkflowExecution、TaskAttempt 和 StageCommit 分别记录工作流、执行尝试与提交结果；最终业务写入与阶段提交在同一短事务内完成，外部模型调用在事务外执行，并通过 epoch 与 fencing 标记阻止旧尝试覆盖新结果。

宏观调度依据容量画像、设备能力、数据位置和剩余预算筛选目标设备，本地 Runtime 再通过原子准入检查当前容量、并发和资源约束；在原子准入成功后，由租约管理器分配并持有 leaseId 与 fencingToken，读取输入数据并执行任务，完成后返回结果、实际配置和物理结算回执。由于租约到期或取消请求不代表计算已经停止，系统还需核对停止状态和资源占用，再决定恢复、重算、等待或终止。

如果一次请求涉及的多项数据更新没有全部成功，系统会先标记该会话为处理中断，暂时隐藏未完成的中间结果，待核对已完成步骤并处理失败步骤后再恢复展示；自动重试和数据补偿机制仍在完善。

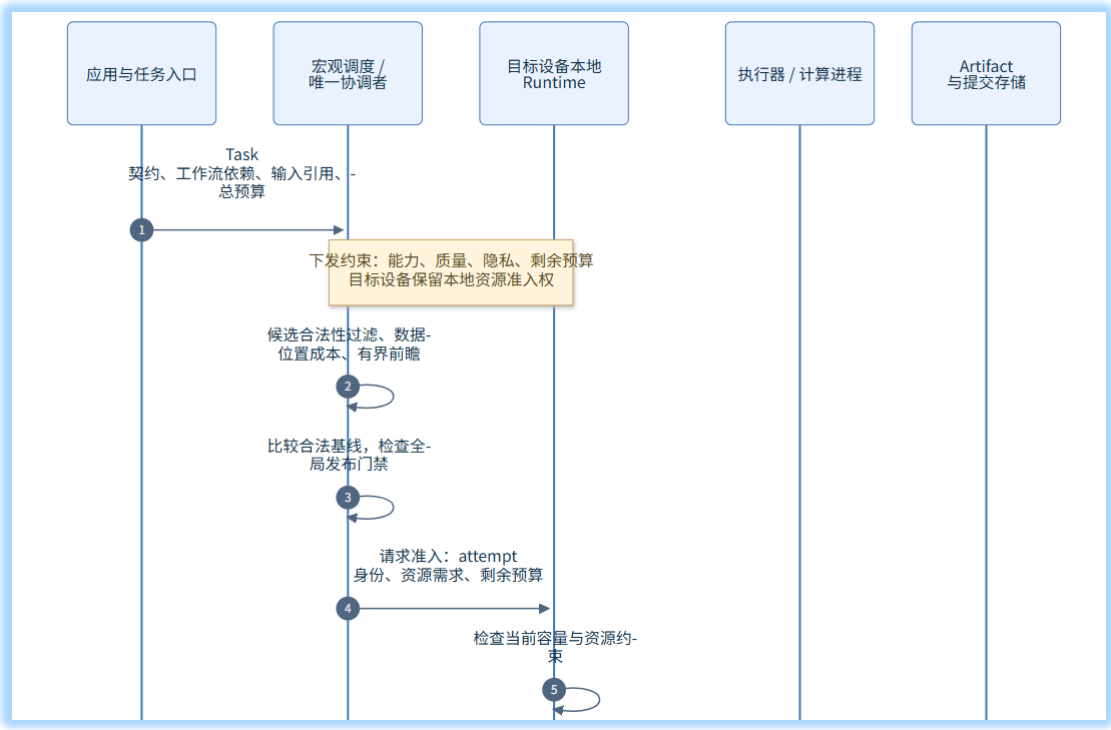


图 4　宏观调度与微观调度协同时序（上）

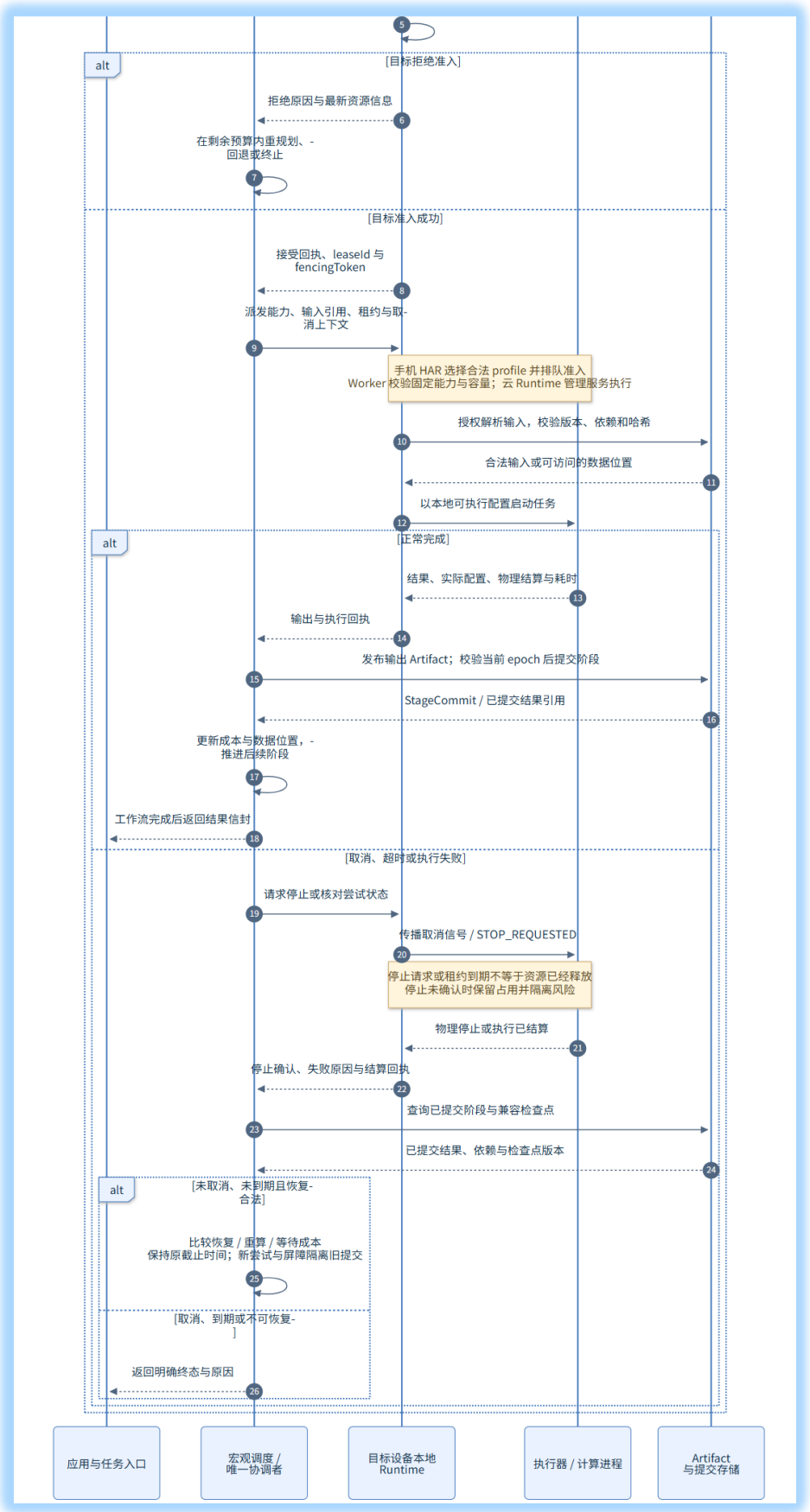


图 4（续）　宏观调度与微观调度协同时序（下）

图中的回环箭头表示组件内部的候选评估、容量检查、重规划和成本更新等步骤。宏观调度根据本地执行反馈持续调整后续阶段，形成“规划—执行—反馈—调整”的协同闭环。

3. 调度算法

智调鸿枢的调度算法为每个任务与阶段联合求解执行设备与执行配置，按“硬约束过滤、成本预测、分层优化、发布门禁、执行反馈”进行分层运算。全局层依据数据位置与端到端成本完成设备放置，本地层依据设备状态在合法配置空间内选择配置；两级共享任务身份与预算，并以真实结果迭代成本模型。

3.1 任务语义与硬约束

任务画像融合能力、模型版本、输入规模、用户等待状态、质量底线、隐私策略与时限。设备状态携带采样时间和来源，过期与未知观测保持显式状态。候选先校验能力、模型及索引空间、数据可达性、质量、权限和资源容量，再参与评分；LOCAL_ONLY 禁止跨机传输。

通过真机实测，我们初步规定设备本地层在可用内存低于 192 MB 或热等级临界时拒绝执行；电量不高于 20% 且未确认充电时，配置策略进入承压保护，排除不适宜承压运行的配置，并采用声明的安全回退。云服务依据服务容量、依赖健康及队列判断准入，避免套用电池阈值。模型兼容性要求种类、维度、数据类型、预处理和索引空间等契约字段一致。

3.2 本地配置效用与队列优先级

执行配置将模型档位、后端、线程、检索维度和批量组合为应用声明的合法配置包，避免独立调参破坏质量边界。对可行配置，效用函数为：

$$U = W + V + I - C_t - R_f - L_q - C_e - R_h - R_m - P_{fb}$$

其中，W、V、I 分别为用户等待、结果可见和业务重要度奖励；其余分项依次为目标时延压力、新鲜度风险、质量损失、相对能耗、热风险、内存风险及反馈惩罚。我们初步设置等待和可见奖励为 30、15，重要度采用归一化值；相对能耗属于配置评分，不作为实测焦耳或续航收益。

$$P = 30w + 15v + 20b + \min(20, 20T_{need}/B_{rem}) + C_p$$

P 为队列优先级评分，w、v 为等待与可见指示量，b 为归一化业务重要度，所需剩余耗时与剩余预算形成工作流紧迫度；目标压力奖励 C_p 上限为 10。评分达到 60、40 分别进入 CRITICAL、HIGH，后台批处理为 LOW，其余为 NORMAL。队列同时提供时间逾期与公平插队：可运行任务等待满 30 秒后，下一空闲槽优先服务其中最早入队者。

3.3 在线成本模型与防震荡

AdaptiveCostModel 在缺少样本时使用任务规模、线程、维度及设备状态修正的保守先验；完成真实执行后更新指数加权均值和偏差。更新式为：

$$\mu_k = 0.75\mu_{k-1} + 0.25t_k$$
$$\delta_k = 0.75\delta_{k-1} + 0.25|t_k - \mu_{k-1}|$$

耗时预测不直接采用平均值，而是取“均值 + max(2×偏差, 15%×均值)”，以偏差提供风险余量，样本越少或波动越大，余量越大，从而避免低估。P50、P95 属于对外指标，需分别满足至少 5 条和 10 条的样本门槛;样本不足时不报告分位数,内部暂定以历史估计兜底。按任务能力、模型、配置、输入规模及状态分桶，避免不同负载互相污染。配置实际执行与计划核对后才能作为可信性能证据，预执行预测与探测数据分别存储。

设备状态恶化允许立即降档，恢复升档受最小保持时间、稳定窗口、合法转移和冷却限制。等待体验、结果效用和设备舒适度分别积累反馈，达到样本阈值后则有界调整对应权重或配置惩罚；响应过慢的反馈不能放宽质量底线。连续失败触发配置熔断，本地与全局策略需独立经过观测、预执行、小流量执行和正式启用四阶段门禁，保留基线与回退操作。

3.4 数据位置感知的有界工作流放置

全局规划须比较实际可执行的基线路径与备选路径。基线可包含云服务，不要求本地设备拥有同等能力。对剩余节点序列，规划成本包含输入读取与传输、排队、模型加载、计算、发布、不确定性和最终结果回传：

$$C(\pi) = \sum_{i \in V_{rem}} (T_{in,i} + T_{q,i} + T_{load,i} + T_{c,i} + T_{pub,i} + \epsilon_i) + T_{return}$$
$$T_{in,i} = \sum_{a \in A_i} \min_{s \in R_a} \{T_{read}(a,s) + T_{link}(a,s,d_i)\}$$

π 表示节点放置序列，剩余节点集合为 V_{rem}；A_i 为节点输入集合，R_a 为数据有效副本所在位置。链路按方向区分吞吐；同设备输入计入读取成本，远端输入计入字节传输与协议开销。已产生或已缓存的副本随候选路径更新，避免重复计算同一输入的跨机传输。有效吞吐已包含的开销不再重复叠加。

$$C(\pi) \le B_{rem}, C(\pi_{base}) - C(\pi_{new}) > C_{switch} + M_{hyst}$$

对候选策略 pi 的切换，系统需同时满足两个条件：新策略的预计成本 C(pi) 不超过剩余预算 B_rem，并且相较当前基线的成本下降幅度 C(pi_base) - C(pi_new) 大于切换开销 C_switch 与迟滞项 M_hyst。为计算这些候选策略的成本，规划器采用有界搜索：每个阶段最

多保留 8 个合法候选，最多向后查看 16 个依赖节点。8 用于限制单个节点参与比较的候选数量，16 用于限制估算 C(pi) 时的前瞻范围，从而避免候选组合过多导致规划本身消耗过多预算。这两个数是当前版本为满足上述成本判断设置的默认搜索上限，不代表系统固定只能支持 8 个候选或 16 个阶段。

4. 智调鸿枢技术方案的关键创新与贡献

4.1 任务语义约束下的完整配置包调度

固定线程或单阈值调度难以表达 AI 任务的质量要求；分别缩减检索维度、候选数和模型档位，还可能叠加产生隐性降质。项目将用户等待、新鲜度、质量底线与资源状态共同建模，把模型、后端、线程、维度和批量作为完整配置包选择，并限制切换范围。执行后核对实际配置，只有满足质量验证与发布门槛的组合才能进入优化路径，具有明确的任务语义和可验证的配置边界。

4.2 融合数据传输成本与预算约束的跨设备前瞻调度

仅比较计算耗时容易将任务卸载到需要反复搬运数据的设备。项目将输入副本、双向链路、排队、冷启动、发布及结果回传纳入同一成本模型，并以有界前瞻评估后续阶段。例如，使用我们独立开发的 SoleAI 进行接入，远端裁剪即使更快，裁剪图回传及后续云阶段仍可能使整条链路变慢，规划器会据此保留云基线。缓存副本随候选路径更新，准备、上传与恢复共用原始预算，避免重复计算传输成本及阶段重试刷新时限。

4.3 跨设备任务取消后的资源释放与结果防覆盖机制

远端调用超时后，旧计算可能仍占资源；直接重派既会放大负载，也可能让迟到结果覆盖新结果。智调鸿枢先检查目标设备的可用容量，通过原子操作预留资源，避免多任务同时抢占同一份容量；任务完成或取消后，需收到执行结束或停止确认回执才释放资源。同时，系统通过 fencing 标记和协调 epoch 校验结果是否来自当前有效的执行尝试，拦截旧任务的迟到结果。最后，阶段结果经校验并提交成功后，才以不可修改的 Artifact 形式供后续阶段读取。

任务恢复首先验证已提交状态、兼容指纹、期限及副作用，随后比较恢复、重算和等待成本。资源占用与结果归属因此得到分别确认，恢复决策也纳入统一成本约束。

4.4 跨越单机边界的多设备资源组织

系统以能力目录、统一任务身份和数据引用构造跨设备资源视图，使应用面向任务提出需求、调度层面向设备群组织执行。全局层选择阶段位置，各设备本地层保留最终准入权；设备能力变化由画像结合观测期限反映到后续决策。

通用调度和业务语义采用双契约，接入不同设备或应用时复用核心策略。这一架构把传统单机资源治理的管理范围拓展到多设备任务协同。

4.5 真实执行与分维度反馈共同校准策略

为探明用户真实偏好，预执行时调度件设置了简易的用户体验反馈。项目将真实分段耗时、配置核对与用户反馈关联到任务身份，等待、结果效用和设备舒适度分别统计；反馈达到门槛后仅作有界校准。预测与实测分开记录，策略按版本经过影子评估和小流量验证，再决定扩大启用范围。该机制使性能学习、质量保护和回退治理形成可审计闭环，减少单次负反馈或不可信样本造成的策略偏移。

（三） 介绍文档(不超过 1500 字)

我们团队在研究现有 OS 调度的过程中发现，传统资源分配存在一些局限。单机调度拘泥于本机，硬件计算资源时常受限，难以同时兼顾 AI 业务需求、用户使用体验等；设备之间在能力与资源上存在差异却难以互相感知，即使附近拥有更适宜的环境与设备，单机调度也无法将其纳入可用范围。这些共性问题不宜由每个应用独立承担，针对这些问题，我们研发了面向鸿蒙生态的 AI 任务调度中间件“智调鸿枢”，把真实的 AI 业务需求融入调度决策，使调度更贴近实际业务场景，从而让用户获得更好的应用使用体验。

项目由调度中间件和我们开发的智能多重识物比价 APP SoleAI 组成。 SoleAI 接入调度件进行多 AI 任务的调度，我们还设计了一个 Web 端调度盘，用以开发时监测调度过程。

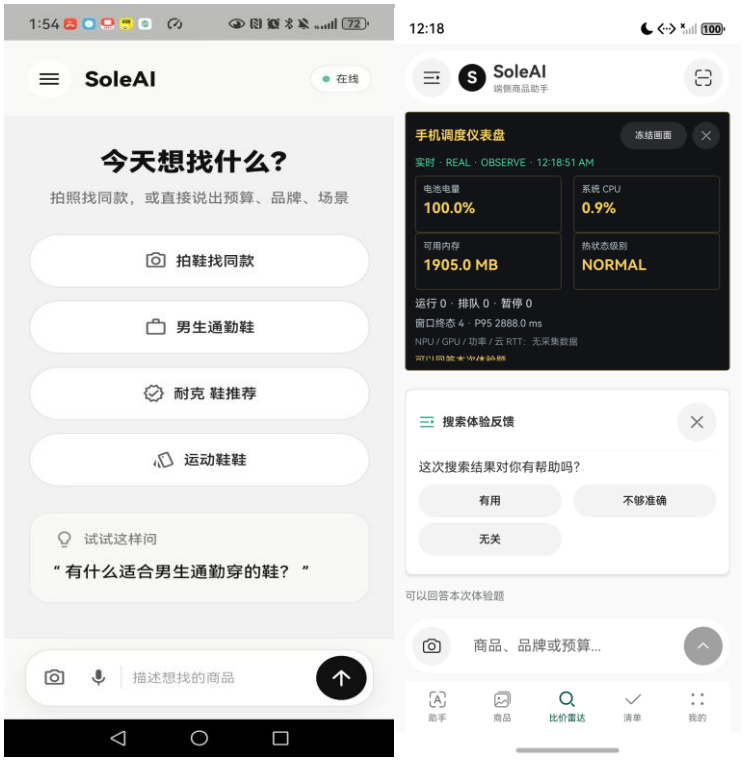


图 5 SoleAI 使用截图

1. 项目优势

业务与设备解耦，便于接入和扩展。应用通过 Task 声明要完成的计算、质量要求和时限，通过 Artifact 引用输入、结果与检查点；商品、会话等业务含义留在应用适配层。接入新业务时，开发者定义任务模板、数据格式并注册执行器；接入新设备时，声明可用能力并实现执行接口。调度核心据此统一决策，避免因针对不同应用和硬件设备而重复编写分配逻辑。

多设备协同调度，充分利用可用计算资源。系统将手机、可信计算节点和云服务纳入统一的能力视图，根据各设备的能力与资源余量，为工作流的不同阶段选择合适的执行位置。本机负载较高、内存不足或持续发热时，可在满足质量、隐私和时限要求的前提下，将适合的阶段交给其他设备处理，让可用算力参与协作，缓解单机资源瓶颈。

宏观统筹与本地调节配合，兼顾协作和资源保护。宏观层根据任务依赖、设备能力、数据位置和剩余预算决定在哪里执行；微观层根据本机负载、内存和热状态，选择可用后端、线程与批量。每个工作流由唯一全局调度器组织，目标设备保留最终准入权。无法接纳时返回原因供宏观层重规划，完成后回传耗时和资源信息，形成持续调整的协同过程。

围绕整条任务链优化，兼顾速度与质量。针对独立调参容易叠加降质的问题，系统把模型档位、线程和批量组合为经过约束的配置包；针对远端计算更快却因反复搬运数据而整体变慢的问题，将传输、排队、加载、计算和回传统一估算，并提前比较后续阶段。切换方案需满足剩余预算且收益覆盖切换开销。真实执行持续修正预测，稳定窗口与冷却机制减少频繁升降档。

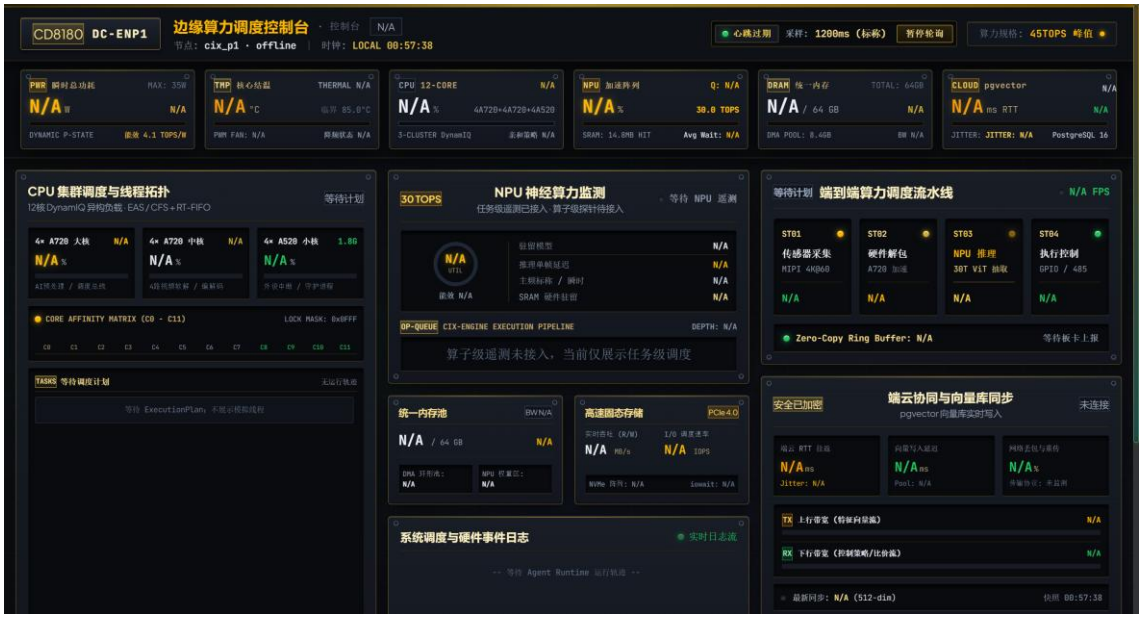


图 6　Web 调度盘界面（离线状态）

2. 应用价值与发展前景

智调鸿枢面向需要端云协作的 AI 应用团队、设备集成商和企业。应用团队可以复用设备选择、任务管理与异常处理能力，把更多精力放在业务体验上；设备提供方可通过适配接口让已有计算资源参与协作。项目可探索以调度 SDK、设备插件和 Web 调度盘提供服务，帮助使用方统一接入、观察和管理多设备任务。

以 SoleAI 为起点，这套能力还可用于文档助理、会议转写、图片整理和批量数据处理。后续将通过任务插件补充各场景的数据规则，并支持按页、时间窗或分块保存进度，让中断任务能够复用已完成结果。这样，更换业务场景时可以延续已有的调度能力，减少从头开发的工作量。

随着任务和设备增多，项目将进一步研究缓存复用、增量检查点与资源配额，减少重复传输，协调多个应用之间的资源使用。跨设备恢复前会检查数据和运行环境是否兼容，再比较恢复与重算的代价。后续将结合小范围试用中的响应体验、资源利用和维护成本，逐步扩大接入场景。

3. 项目局限

跨设备传输受带宽、时延和数据量影响，实时任务更适合链路稳定的局域网或近端环境，远程协作则需权衡传输开销与计算收益。此外，新设备仍需适配接口，大规模并发下的调度效果有待验证。

参考资料

[1] GUJARATI A, KARIMI R, ALZAYAT S, et al. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up[C]//OSDI 2020. 2020: 443–462. https://www.usenix.org/conference/osdi20/presentation/gujarati

[2] CRANKSHAW D, SELA G E, ZUMAR C, et al. InferLine: ML Prediction Pipeline Provisioning and Management for Tight Latency Objectives[EB/OL]. arXiv:1812.01776, 2020. https://arxiv.org/abs/1812.01776

[3] WANG Z Z, MAO J, FRIED D, et al. Agent Workflow Memory[EB/OL]. arXiv:2409.07429, 2024. https://arxiv.org/abs/2409.07429

[4] LIU Y, SRA M, INALA J P, et al. ReUseIt: Synthesizing Reusable AI Agent Workflows for Web Automation[EB/OL]. arXiv:2510.14308, 2026. https://arxiv.org/abs/2510.14308

[5] OpenHarmony. HTTP SDK 接口定义（@ohos.net.http）[EB/OL]. https://raw.githubusercontent.com/openharmony/interface_sdk-js/master/api/@ohos.net.http.d.ts

[6] 华为. SystemCapability（系统能力）[EB/OL]. https://developer.huawei.com/consumer/cn/doc/harmonyos-references-V14/js-apis-syscap-V14

[7] 华为. HarmonyOS 应用开发：分布式任务调度[EB/OL]. https://developer.huawei.com/consumer/cn/codelab/HarmonyOS-hap5/index.html

[8] The Linux Kernel. Control Group v2[EB/OL]. https://docs.kernel.org/admin-guide/cgroup-v2.html

[9] The Linux Kernel. Deadline Task Scheduling[EB/OL]. https://docs.kernel.org/scheduler/sched-deadline.html

[10] The Linux Kernel. EEVDF Scheduler[EB/OL]. https://docs.kernel.org/scheduler/sched-eevdf.html

[11] The Linux Kernel. Capacity Aware Scheduling[EB/OL]. https://docs.kernel.org/scheduler/sched-capacity.html

[12] Google. LiteRT 官方文档[EB/OL]. https://developers.google.com/edge/litert

[13] Google AI Edge. LiteRT-LM 项目文档[EB/OL]. https://github.com/google-ai-edge/LiteRT-LM

[14] LangChain. LangGraph Overview[EB/OL]. https://docs.langchain.com/oss/javascript/langgraph/overview

[15] LiteLLM. Router — Load Balancing[EB/OL]. https://docs.litellm.ai/docs/routing

[16] Prisma. Prisma Migrate Limitations and Known Issues[EB/OL]. https://www.prisma.io/docs/orm/v6/prisma-migrate/understanding-prisma-migrate/limitations-and-known-issues

[17] pgvector. Querying[EB/OL]. https://github.com/pgvector/pgvector#querying

[18] 阿里云. 什么是抢占式实例[EB/OL]. https://help.aliyun.com/zh/ecs/user-guide/what-is-a-spot-instance

[19] 腾讯云. 竞价实例（波动型）[EB/OL]. https://cloud.tencent.com/document/product/213/17816

二、格式及其他要求

（一）字体：宋体

（二）字号

1、标题：二号，粗体

2、作者：三号，粗体

3、一级标题：三号，粗体

二级标题：四号，粗体

三级标题：小四号，粗体

四级标题：五号，粗体

4、正文：五号

（三）行距：单倍行距

（四）页面设置

1、页边距：上：2.5 厘米　下：2.5 厘米

左：3 厘米　　右：3 厘米　装订线：0 厘米

2、页眉：1.5 厘米

页脚：1.5 厘米

3、纸型：A4，纵向

（五）插入页码

位置：页面底端

对齐方式：外侧

（六）其他

1、PDF 格式提交，命名格式为 01-作品说明文档+参赛队伍名称

2、文档中若包含插图、表格请按序编号并命名

3、主体内容不超过 20 页（参考资料及附录不计入总页数）